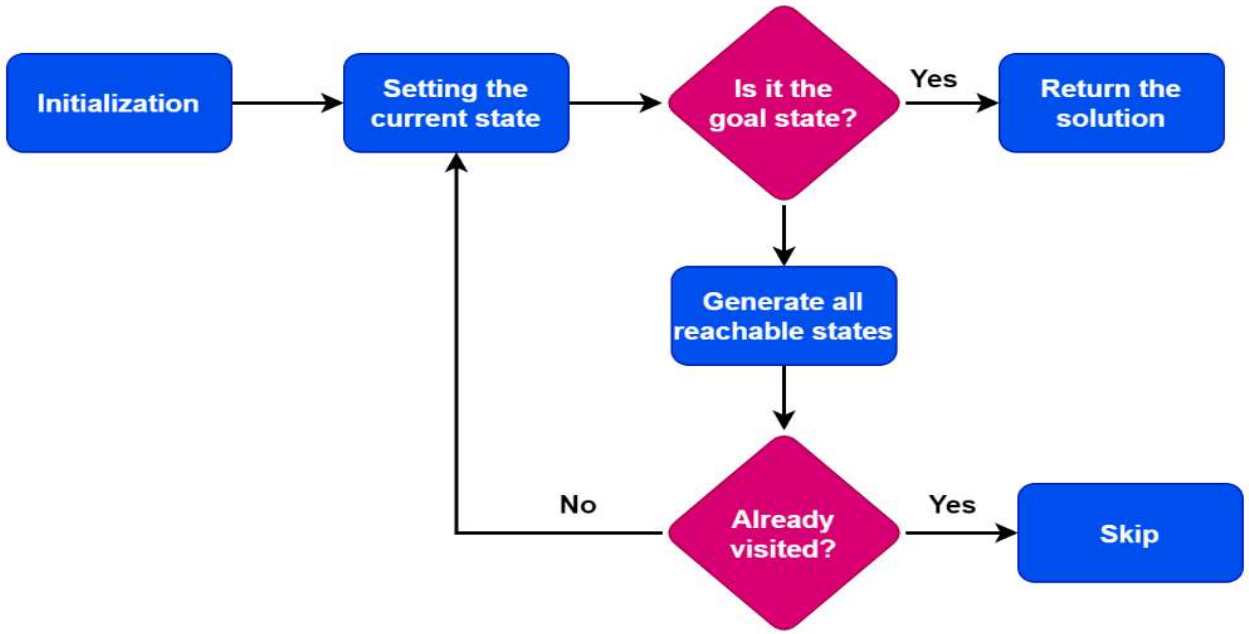
Unit -3

Introduction to State Space and State Space Search

State Space Search

- State space search is a method which is used to find a solution to a problem by searching through the set of possible states of the problem.
- The steps of a state space search algorithm:



The first step is to initialize the search by setting the initial state as the current state. After the initialization step, check if the current state is a goal state. Finally, if the current state is a goal state, terminate the algorithm and return the result.

However, if the current state is not the goal state, generate the set of possible states that can be reached from the current state. Additionally, these states are known as successor states.
Furthermore, for each successor state, check if it has been previously visited. If the state is already explored, skip the state. If it has not been visited, add it to the queue of states to be visited.

Moving forward, set the next state in the queue as the current state and check if it's a goal state. If the goal state is find, then return the result. Otherwise, repeat the previous step until the goal state is not find or finish exploring all the states.

Types of search algorithms:-

Based on the search problems we can classify the search algorithms into
1. Uninformed (Blind search) search
2. Informed search (Heuristic search) algorithms.

Parameters	Informed Search	Uninformed Search
Known as	It is also known as Heuristic Search.	It is also known as Blind Search.
Using Knowledge	It uses knowledge for the searching process.	It doesn't use knowledge for the searching process.
Performance	It finds a solution more quickly.	It finds solution slow as compared to an informed search.
Completion	It may or may not be complete.	It is always complete.
Cost Factor	Cost is low.	Cost is high.
Time	It consumes less time because of quick searching.	It consumes moderate time because of slow searching.
Direction	There is a direction given about the solution.	No suggestion is given regarding the solution in it.
Implementation	It is less lengthy while implemented.	It is more lengthy while implemented.
Efficiency	It is more efficient as efficiency takes into account cost and performance. The incurred cost is less and speed of finding solutions is quick.	It is comparatively less efficient as incurred cost is more and the speed of finding the Breadth-Firstsolution is slow.
Computational requirements	Computational requirements are lessened.	Comparatively higher computational requirements.
Size of search problems	Having a wide scope in terms of handling large search problems.	Solving a massive search task is challenging.
Examples of Algorithms	•Greedy Search •A* Search •AO* Search •Hill Climbing Algorithm	•Depth First Search (DFS) •Breadth First Search (BFS) •Branch and Bound

8 Puzzle Problem

It has set off a 3x3 board having 9 block spaces out of which 8 blocks having tiles bearing number from 1 to 8. One space is left blank. The tile adjacent to blank space can move into it. We have to arrange the tiles in a sequence for getting the goal state.

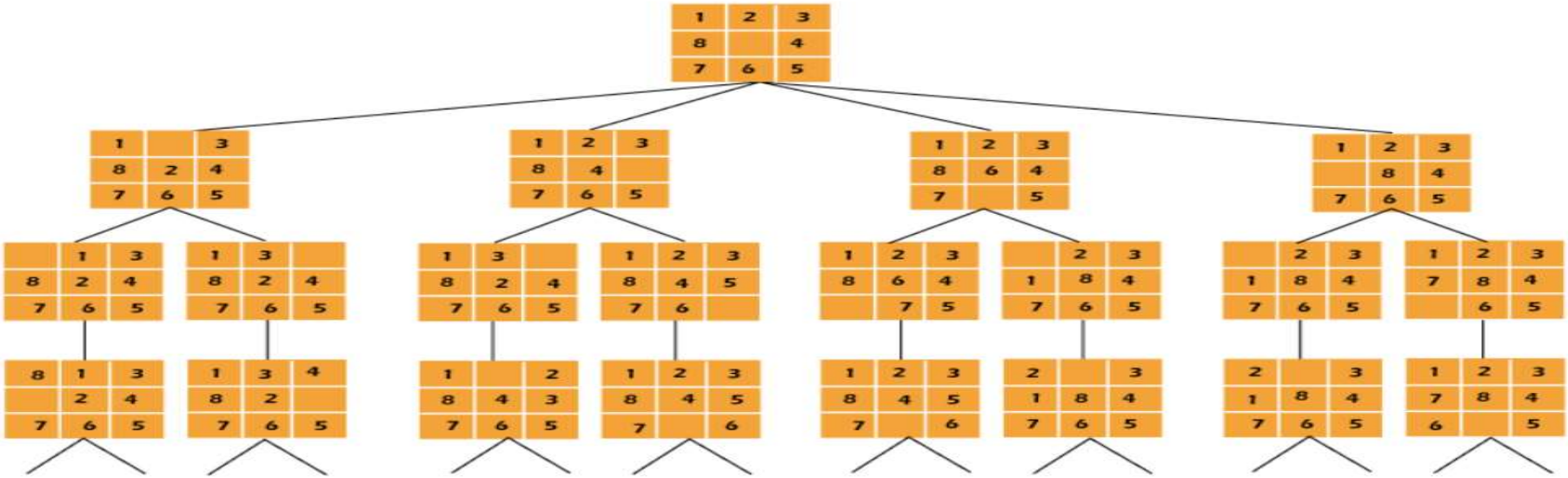
Initial Configuration

1	2	3
5	6	
7	8	4

Final Configuration

1	2	3
5	8	6
	7	4

8 Puzzle Problem using Brute Force approach



<u>Time complexity:</u> In worst case time complexity in **BFS is O(b^d) know as order of b raise to power d.** In this **particular case it is (3^20).**

b-branch factor

d-depth factor

1. Euclidean Distance represents the shortest distance between two points.

2. Manhattan Distance is the sum of absolute differences between points across all the dimensions.

8 Puzzle Problem using Heuristic Approach

Initial state

1	2	3
	4	6
7	5	8

Goal state

1	2	3
4	5	6
7	8	

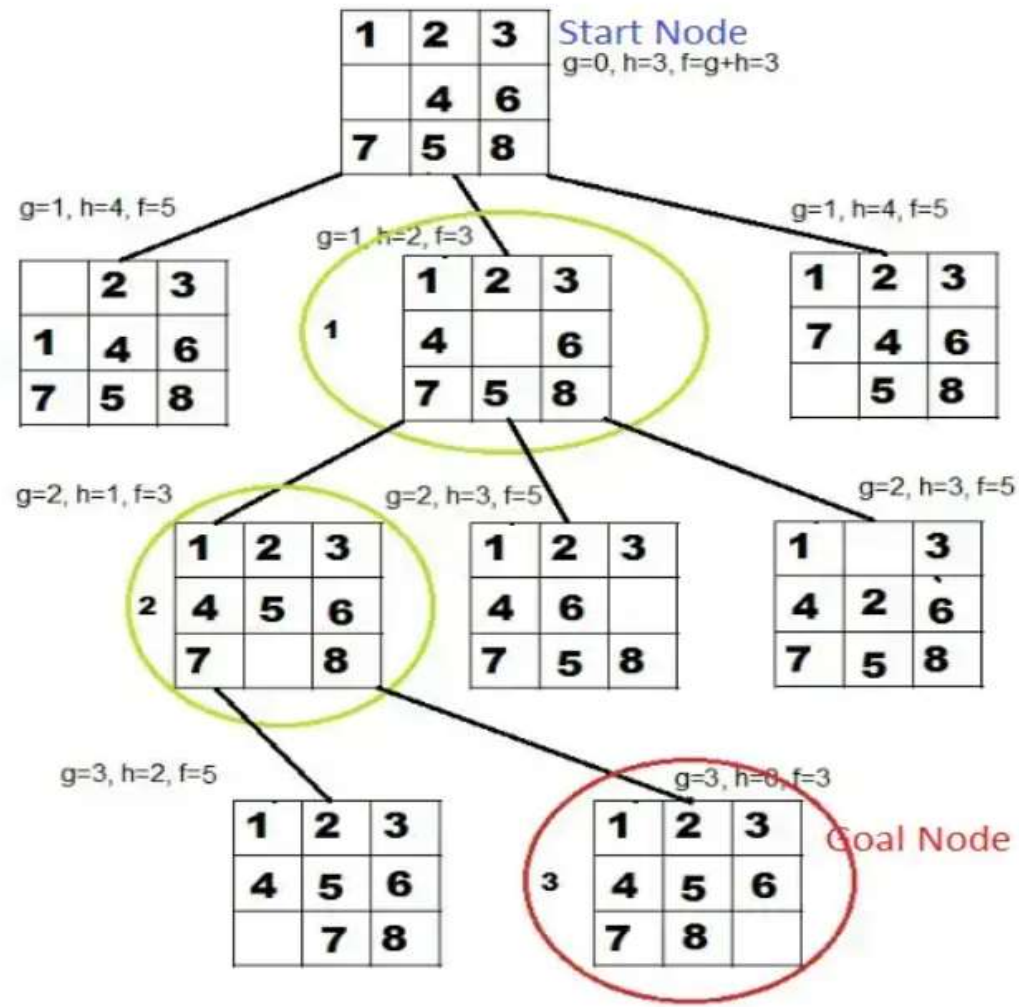
Costs function :

f(x) = g(x) + h(x)

where

g(x) = the costs of reaching to the current node from the root. Number of Step

h(x) = the costs of reaching to an answer node from the x. Total miss placed tiles



Hill Climbing Algorithm in Artificial Intelligence

- Hill climbing is a simple optimization algorithm used in Artificial Intelligence (AI) to find the best possible solution for a given problem.
- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.

Features of Hill Climbing:

Following are some main features of Hill Climbing Algorithm:

•**Generate and Test variant:** Hill Climbing is the variant of Generate and Test method.

- Generate possible solutions.
- Test to see if this is the expected solution.
- If the solution has been found quit else go to step 1.

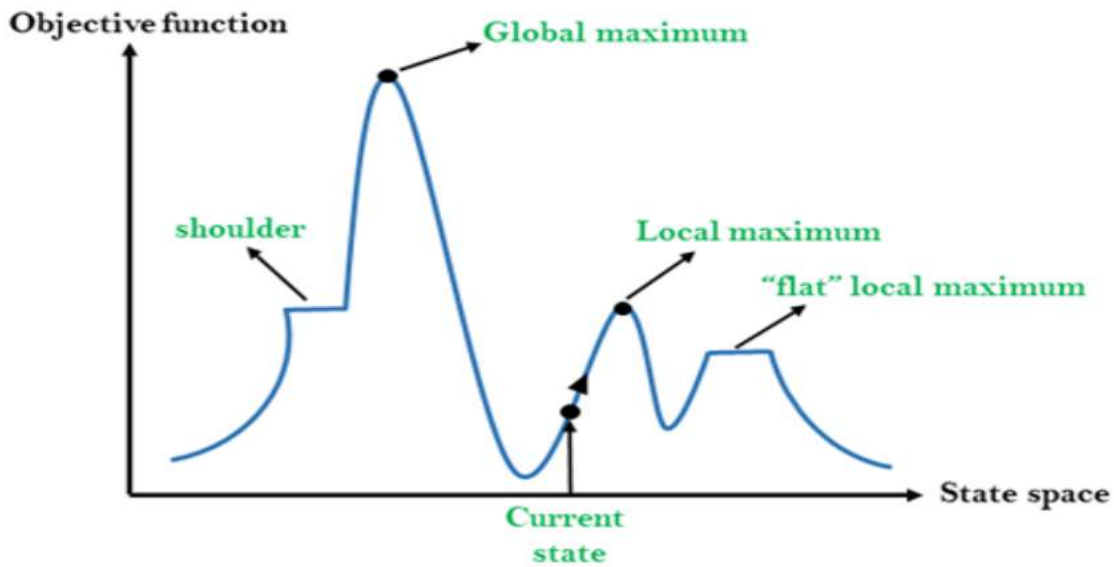
•**Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.

•**No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

State-Space Diagram for Hill Climbing:

The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.

- On **Y-axis** we have taken the function which can be an objective function or cost function,
- state-space on the **x-axis**.
- If the function on Y-axis is **cost** then, the goal of search is to find the global **minimum** and local minimum.
- If the function of Y-axis is **Objective** function, then the goal of the search is to find the global **maximum** and local maximum.



- **Local Maximum:** Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

- **Global Maximum:** Global maximum is the best possible state of state space landscape. It has the highest value of objective function.

- **Current state:** It is a state in a landscape diagram where an agent is currently present.

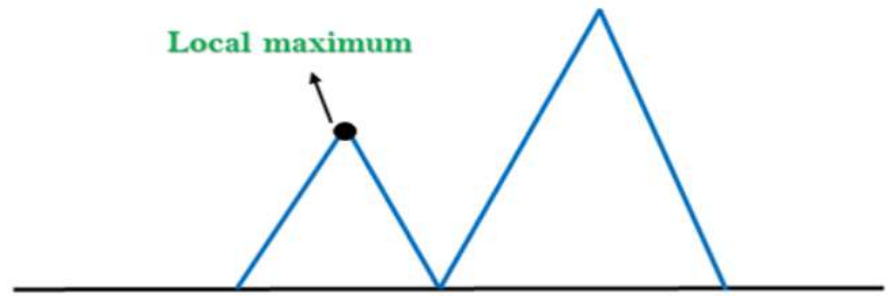
- **Flat local maximum:** It is a flat space in the landscape where all the neighbor states of current states have the same value.

Problems in Hill Climbing Algorithm

1. Local Maximum
2. Plateau
3. Ridge

1. Local Maximum

The algorithm terminates when the current node is local maximum as it is better than its neighbours. However, there exists a global maximum where objective function value is higher

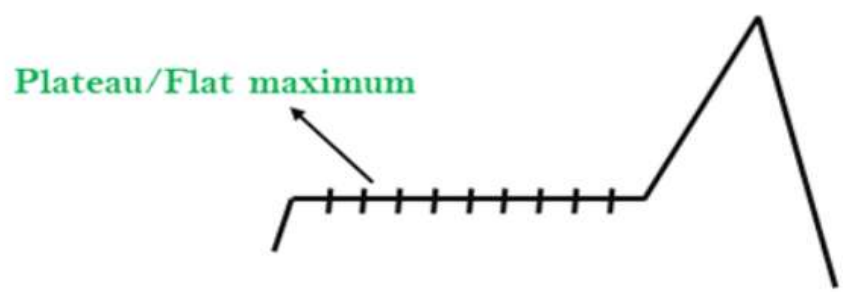


2. Ridge

Ridge occurs when there are multiple peaks and all have the same value or in other words, there are multiple local maxima which are same as global maxima

3. Plateau

Plateau is the region where all the neighbouring nodes have the same value of objective function so the algorithm finds it hard to select an appropriate direction.



Simple Hill Climbing:

Simple hill climbing is the simplest way to implement a hill climbing algorithm. **It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state**. It only checks it's one successor state, and if it finds better than the current state, then move else be in the same state.

Algorithm for Simple Hill Climbing:

•**Step 1:** Evaluate the initial state, if it is goal state then return success and Stop.
•**Step 2:** Loop Until a solution is found or there is no new operator left to apply.
•**Step 3:** Select and apply an operator to the current state.
•**Step 4:** Check new state:

- If Current State = Goal State, Return success and exit
- Else if New state is better than current state then Goto New state
- Else if not better than the current state, then return to step2.

•**Step 5:** Exit.

Steepest-Ascent hill climbing:

The steepest-Ascent algorithm is a variation of simple hill climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors

Algorithm for Steepest-Ascent hill climbing:

•**Step 1:** Evaluate the initial state, if it is goal state then return success and stop, else make current state as initial state.
•**Step 2:** Loop until a solution is found or the current state does not change.

- Let SUCC be a state such that any successor of the current state will be better than it.
- For each operator that applies to the current state:
  - Apply the new operator and generate a new state.
  - Evaluate the new state.
  - If it is goal state, then return it and quit, else compare it to the SUCC.
  - If it is better than SUCC, then set new state as SUCC.
  - If the SUCC is better than the current state, then set current state to SUCC.

•**Step 5:** Exit.